# Deep Learning

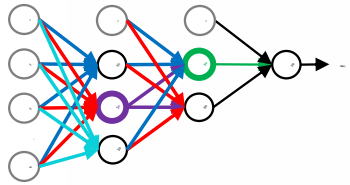
# 9

Pre

## CS 3244
## Machine Learning

NUS | Computing

# Recap and Forecast



The Basics of Neural Networks (ANN or NN):

- Explain how neural networks combine perceptrons;
- Explain the topology of a NN;
- Given the index of a weight, find it in a graphical representation of a NN (and vice versa).

- Name potential problems with NNs;
- Describe how a neural network is trained using the backpropagation algorithm;
- Name NN regularization techniques;
- Obtain practical experience constructing NNs using PyTorch

Learning outcomes for this week:

- Understand why deep learning was enabled by certain key technological milestones;
- Understand the Recurrent and Convolutional NN architectures (shortened as RNNs / CNNs);
- Understand how DNNs regularize and optimize via dropout, pretraining and choice of activation function;
- Obtain practical experience in constructing DNNs with PyTorch.

# Deep Learning

## CS3244 Machine Learning

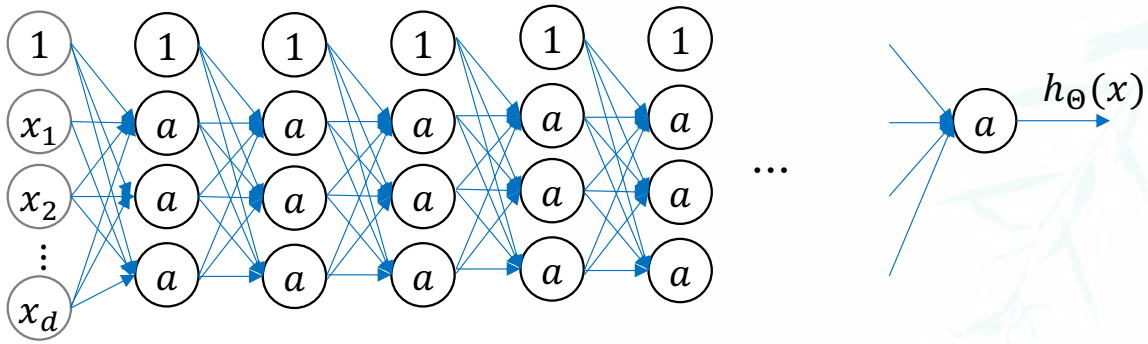
NUS National University of Singapore
Department of Computer Science
School of Computing

# Deep Learning

Deep – Many hidden layers of models stacked upon each other

Ensembling (Week 11): Feed $h_1 \ldots h_T$ into a learning model $\mathcal{A}$ as input

Learning – Usually connotes the use of Neural Networks, although can be used with other models.




How deep is deep?
More than 2, but commonly now 100s or more layers.

# Theoretical Advantages of Deep Representation

$f(x)$ that can be compactly represented by depth $(l)$ network might require an exponentially large number of nodes in a depth $(l-1)$ architecture. (Hastad Proof)

Consequences:

- Computational: We don't need exponentially many elements in the layers
- Statistical: poor generalization might occur when using an insufficiently deep architecture



Slide Credits: Aarti's CMU SlidesBlog

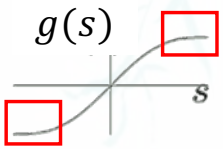
# A history of stumbling blocks

Multi-level ANNs have been around for a long time, since the first biologically-inspired neuron in the 1960s.

So why weren't they popular until recently?

Some key problems:

- Computational complexity – Tons of computation
- Vanishing gradient – Traditional error gradients "flat-line" at their limits
- Other good shallow models worked well – SVM



# The perfect storm

## Big Data

More raw data – infeasible to create features by hand, need to generate features unsupervised

## Hardware Support

Multicore GPUs – memory bandwidth & parallelism (i.e., linear algebra): ~10x faster

## Algorithms and Concepts

*Good training algorithms for many-layer networks*

1. Initialization – Pre-training / Autoencoding
2. Dropout – Regularization
3. ReLU – to address vanishing exploding gradients

# DNN Overview

## CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

# Warning

Deep learning is an area with disruptive change within the field, primarily accelerated by the faster pace of science achieved by arXiv and github.

Much of the material that in these slides will change soon after you learn it.

Try your best not to overfit your own model of deep learning and generalize to the concepts that the implementation and mathematics are trying to achieve.

# Key Idea: More expressiveness with less parameters

Recall: *Bias-Variance Tradeoff*

Or *VC Dimensionality* if you know that from SVM theory.

Networks of individual units add flexibility (less bias error)

But if we add connections, generalization will suffer (more parameters, thus more variance error)

Want to choose model representations that are likely to fit the true signal.

Find representations that embody good representations.

# Alphabet Soup

- Convolutional (Neural) Network (CNN)
  Constrained NN for local features: esp. images & audio

- Recurrent Neural Network (RNN)
  Sequential, time-series data; forecasting, language
  Allow NN outputs as inputs

- Recursive Neural Network (RNN)
  Structured data: (parsed) language structure

- Generative Adversarial Network (GAN)
  Training a network to be more discriminative.

Image Credits: Wikimedia Commons

# Layer Cake and Activation Zoo

- Dense / Fully Connected (FC)
- (Max) Pooling / Subsampling
- Convolution
- Non-linear Activation
- Attention
- Residual

- Sigmoid
- Hyperbolic Tangent
- ReLU
- Leaky ReLU
- Maxout

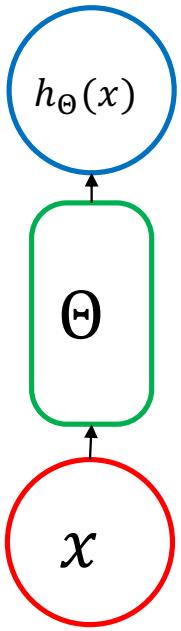
- Softmax
- Hierarchical Softmax

# Recurrent NNs

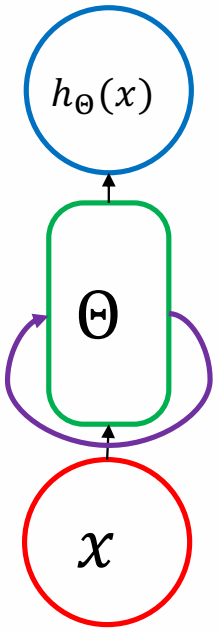
## CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

# Recurrence: Adding Serial Dependencies




Standard, non-recurrent NN



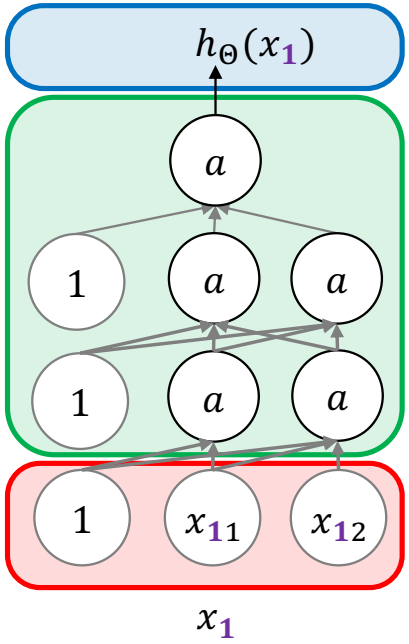

Recurrent NN



Adopted from section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

# Unrolling the Recurrence

(Usually) adding an edge between a unit and a copy of itself at the next time step.

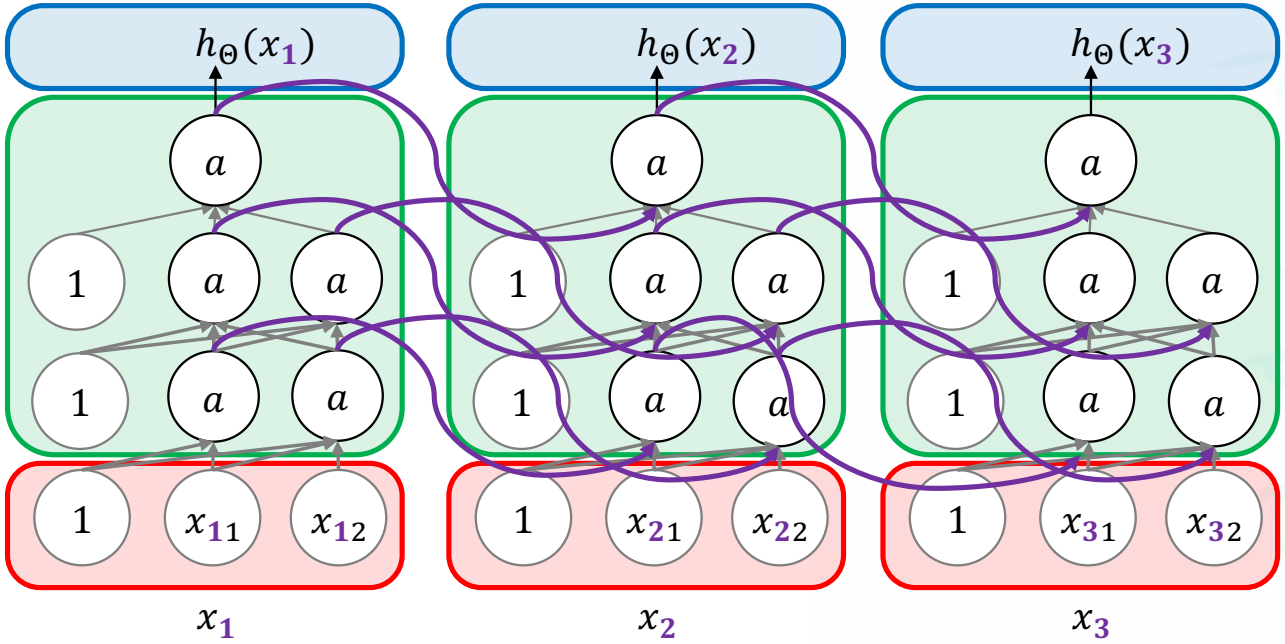





Adopted from section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

# Unrolling the Recurrence

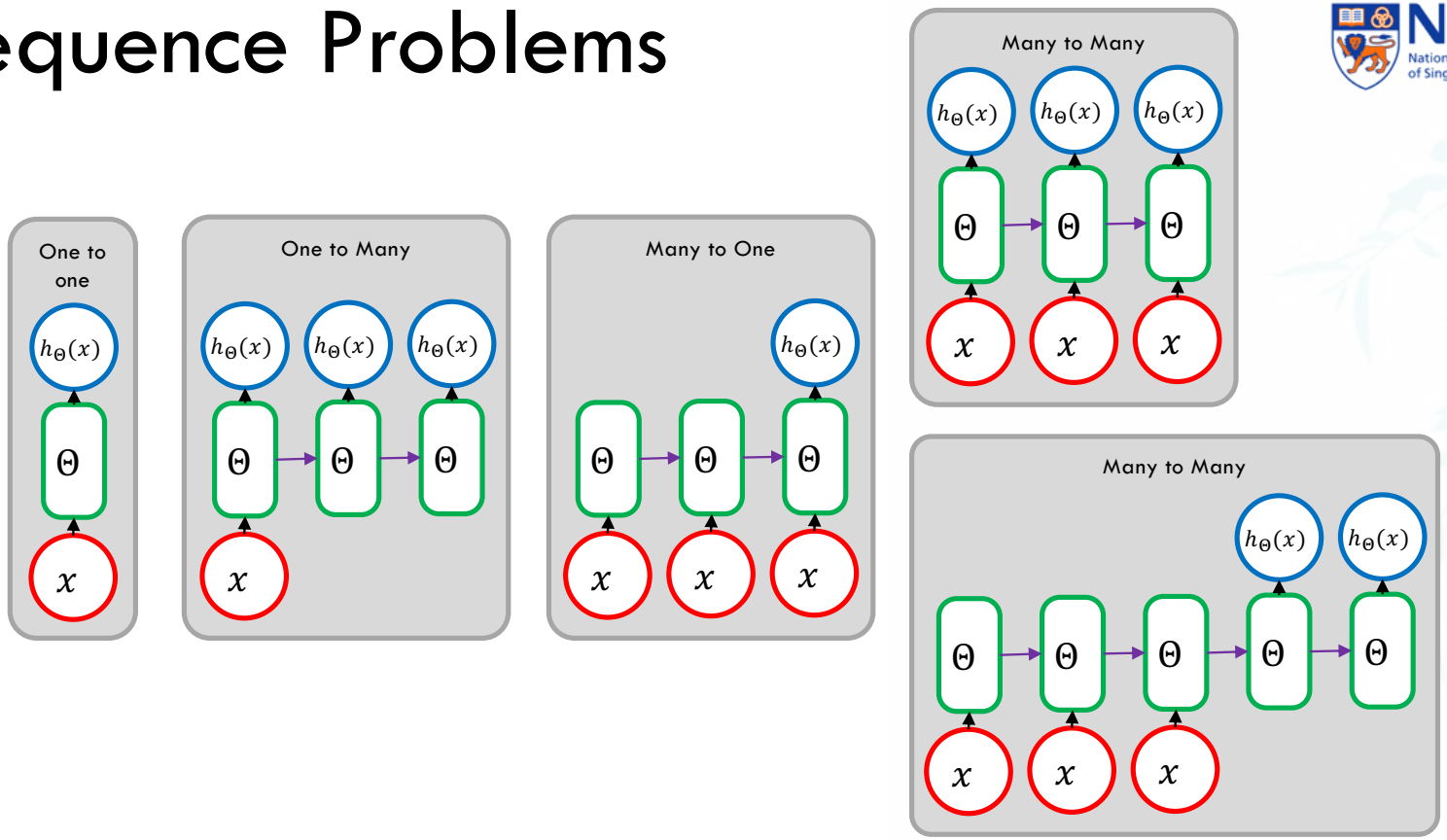
(Usually) adding an edge between a unit and a copy of itself at the next time step.






Adopted from section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

# Sequence Problems




Adopted from section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

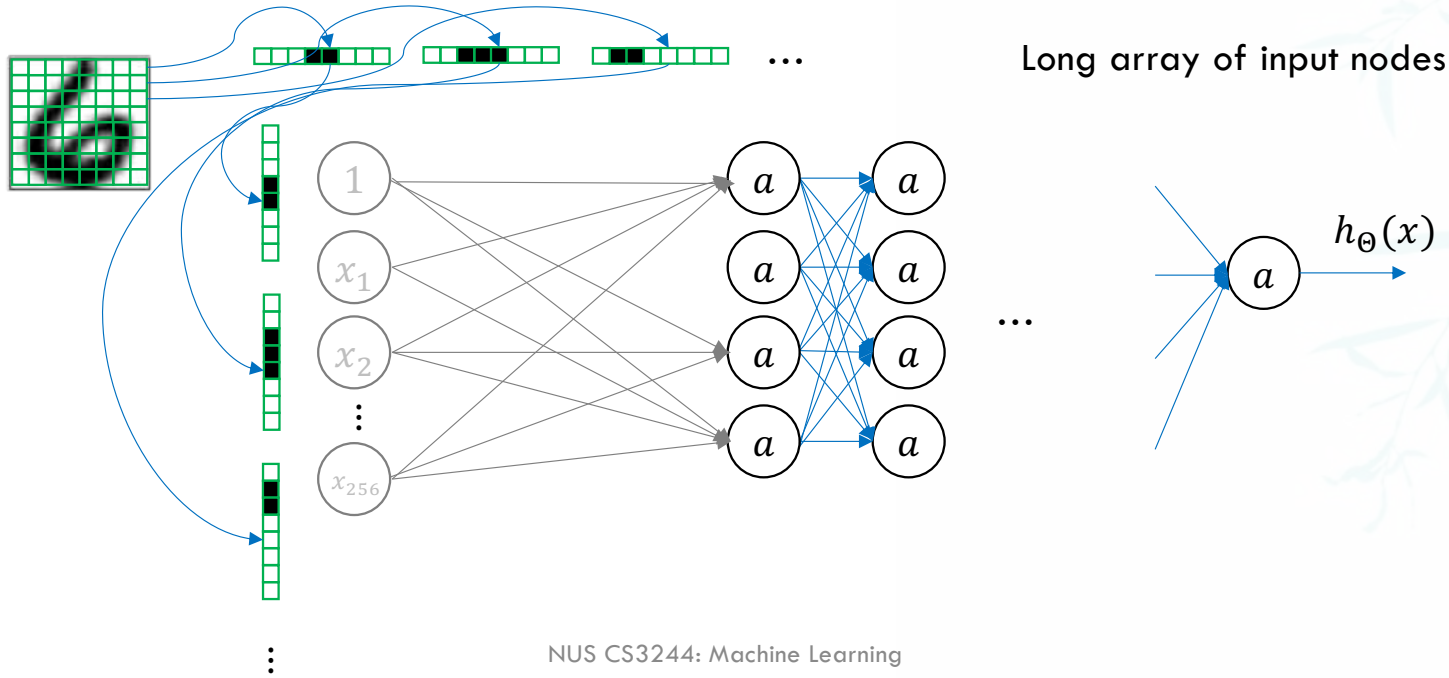
# Convolutional NNs

## CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

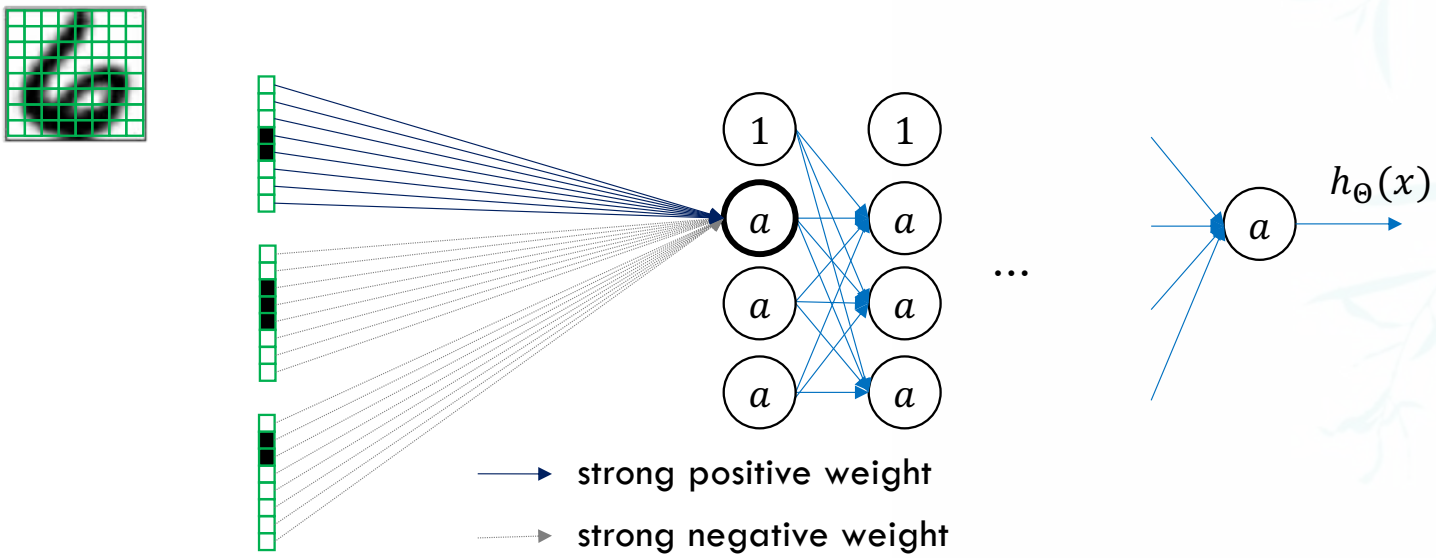
# What is it doing?

Easiest visualized, so let's turn to the vision problem MNIST



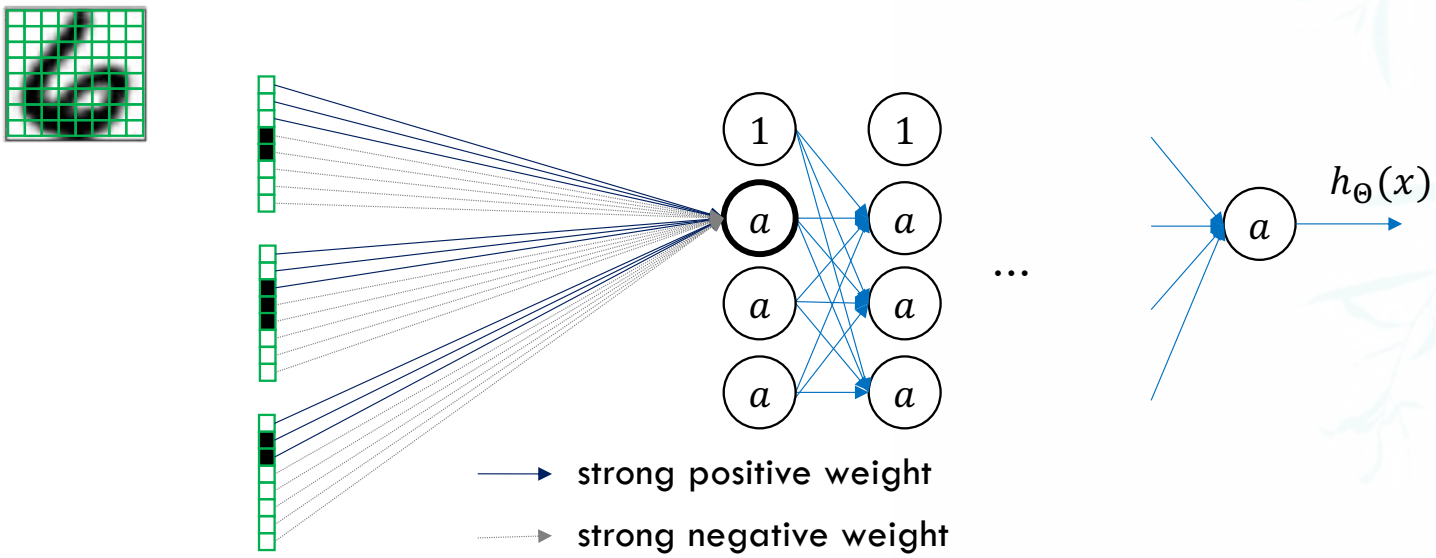
# What is it doing?

Quick Quiz 1 Answer: _____



# What is it doing?

Quick Quiz 2 Answer: _____



# Low Level Features – Convolution

Traditional Image Processing Convolution Filters

Take a look at http://setosa.io/ev/image-kernels/

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |

Kernel

| | | | | |
|---|---|---|---|---|
| $1_{\times 1}$ | $1_{\times 0}$ | $1_{\times 1}$ | 0 | 0 |
| $0_{\times 0}$ | $1_{\times 1}$ | $1_{\times 0}$ | 1 | 0 |
| $0_{\times 1}$ | $0_{\times 0}$ | $1_{\times 1}$ | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |

Image

| | | |
|---|---|---|
| 4 | | |
| | | |
| | | |

Convolved Feature

ıge credits: Nvidia Blog

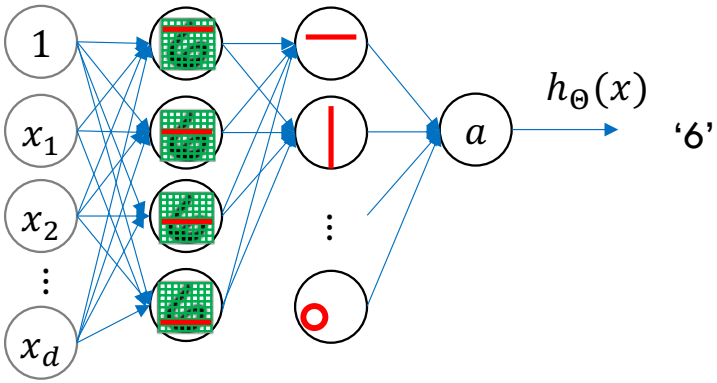
# Low Level Image Features

- Vertical Line Detector
- Horizontal Lines
- Small Circles

… but in specific places

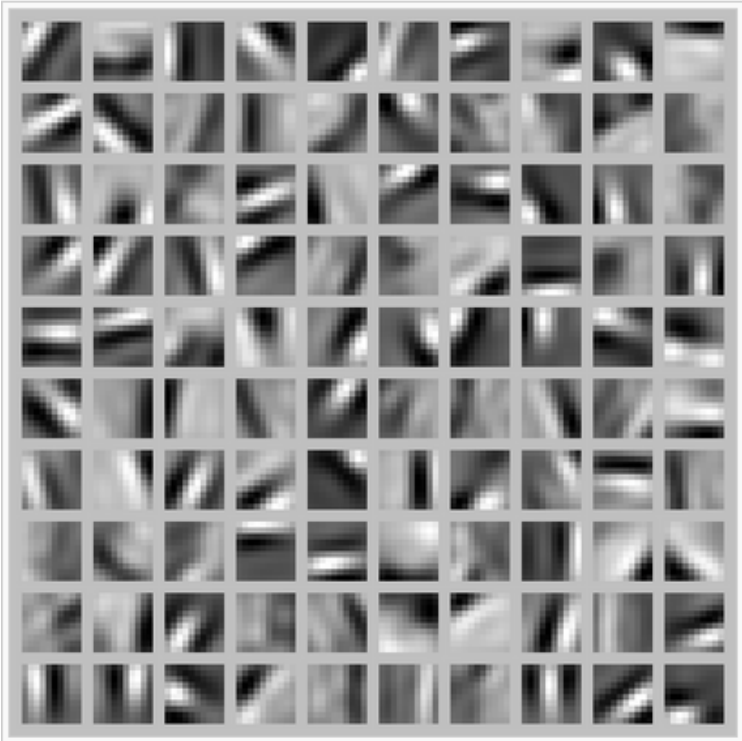
# From low to high level features

Positional and rotational invariance might be achieved at a higher level neuron



# Unsupervised Learned Features

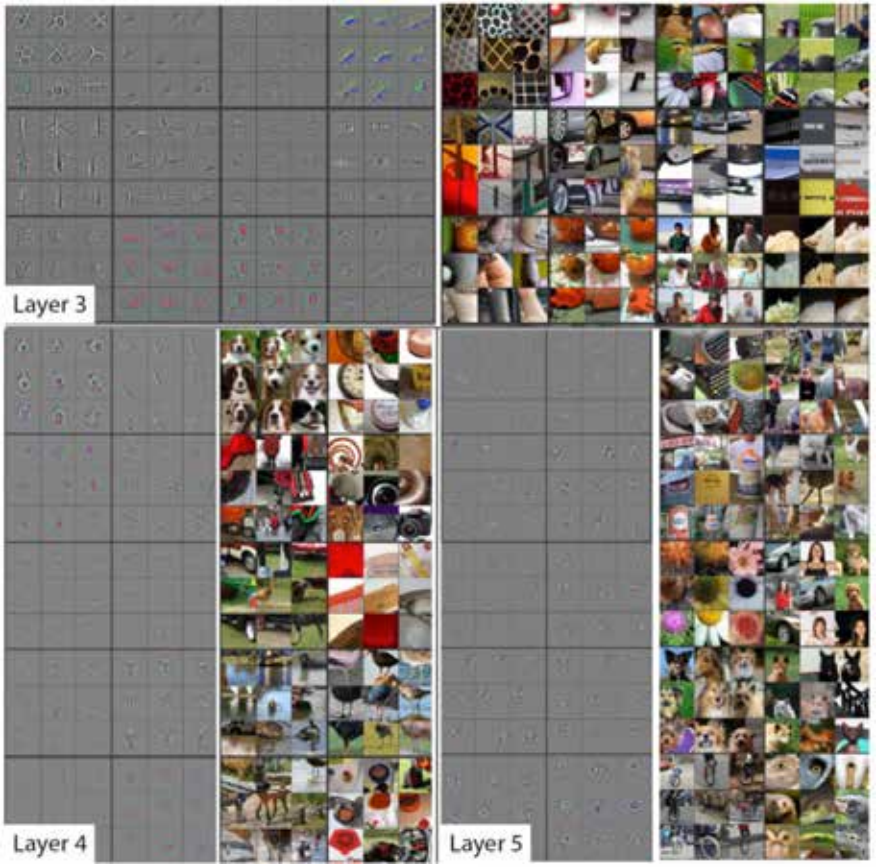
Best when input space is locally structured – spatially or temporally: in images and language



Each square in the figure shows the input image that maximally activates one unit

Image credits: Stanford Deep Learning Wiki

# ConvNet Layers 3–5






Slide credits: Zeiler and Fergus (2013)

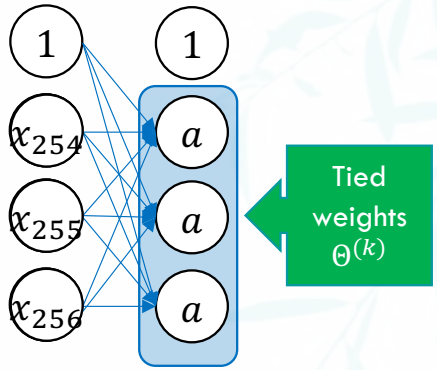
# Convolutional Neural Network (CNN)

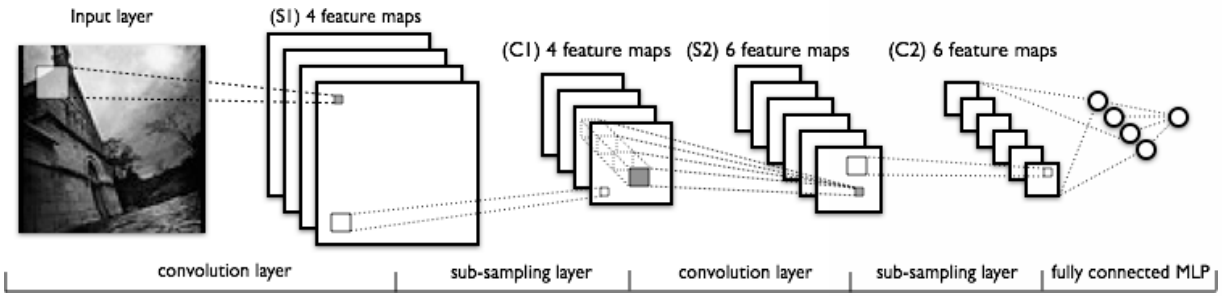
A special case of the general multilayer perceptron (MLP) network, restricted to have fewer parameters.

Applied to images where features are local.

A feature map applies the same learned filter (convolution) over the entire image field

Input layer is not fully connected, but just connected for a particular input patch.



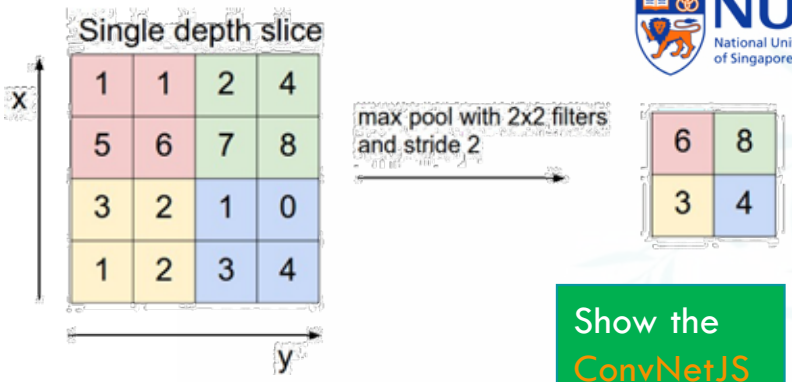



Slide Credits: LeNet Tutorial

# Pooling

Pooling downsamples data resolution, helping generalization.

Typically interleaved between convolution layers.




Show the ConvNetJS demo

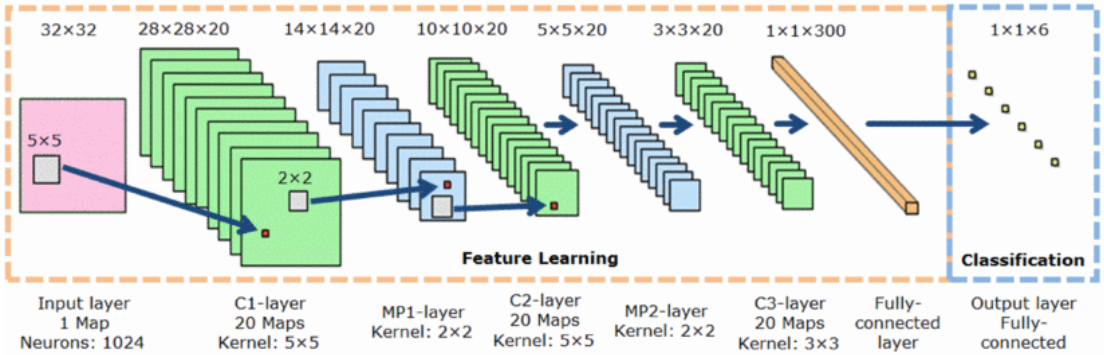



Image credits: SciePub and Stanford CS 231N

# Training DNNs

## CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

# Training Deep NNs

The usual: SGD with backprop for gradient determination.

We notice hidden layers closer to the input have smaller activations.

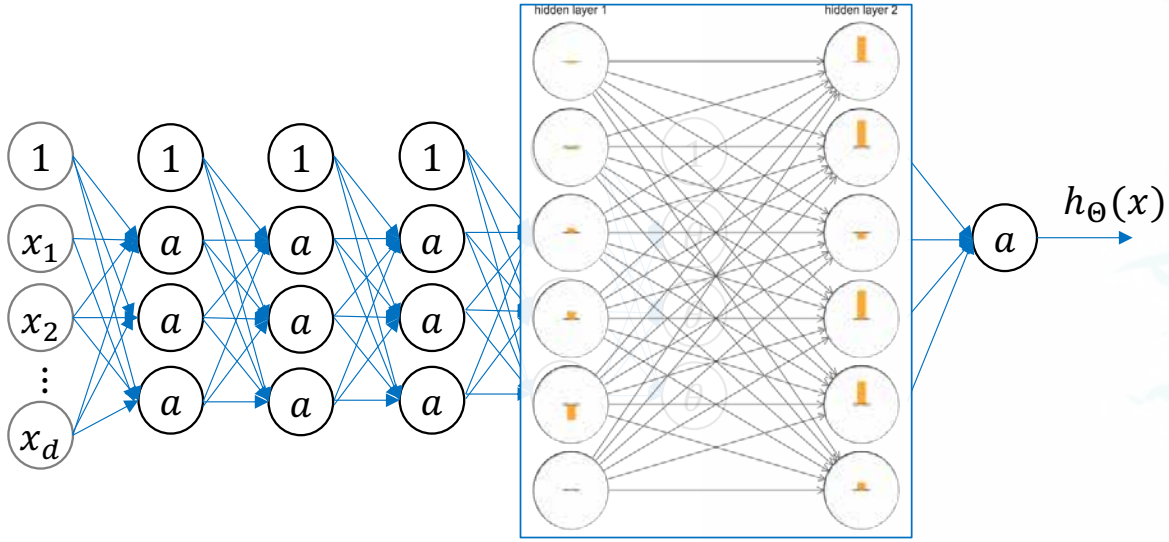


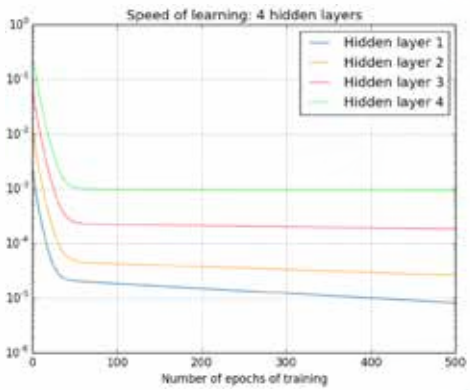


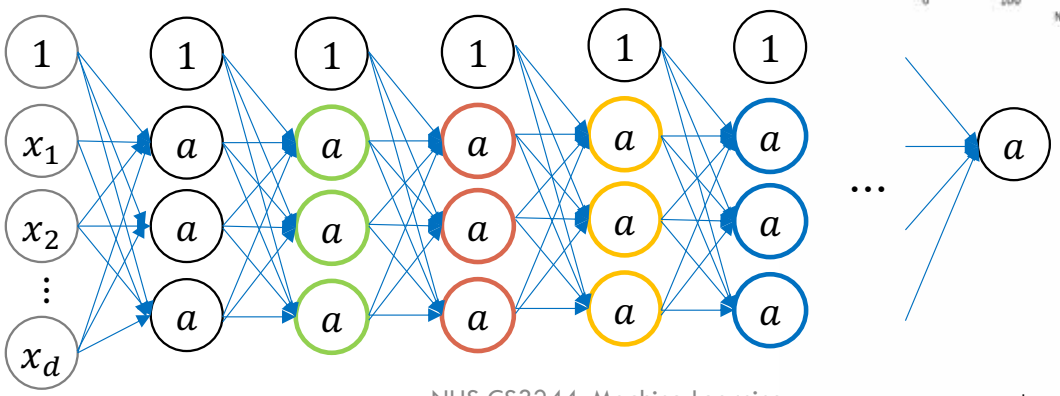

Image Credits: Nielsen's Neural Networks Website

# Vanishing (Exploding) Gradients

This phenomenon gets more pronounced the further back you go in the DNN.

Earlier layers take magnitudes more time to train.



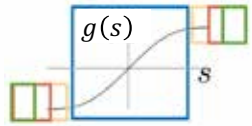
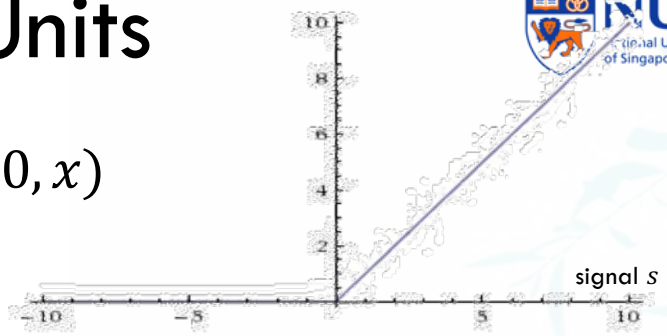






Image Credits: Nielsen's Neural Networks Website

# ReLU: Rectified Linear Units

$$g(x) = \max(0, x)$$




Replaces tanh and sigmoid to avoid such gradient problems within hidden layers where it's not necessary to have probabilistic interpretation

- Not subject to saturation problem, where the gradient is flat (almost zero)
- Easy computation: no exponential operations
- Doesn't have second or higher order terms in cost function

- Sensitive to learning rate

# Unsupervised Pre-Training

Issue: Initialization is critical for DNN.

Use unsupervised learning to set the network going to encode useful features for supervision.

Subsequent supervised learning said to fine-tune results.

Regularization perspective: Representations good $P(x)$ for are good for $P(y|x)$ too.

# Sparse Autoencoders

Try to learn $y_j^{(L)} = x_j^{(0)}$ by way of a compressed representation of the inputs.

Similar in spirit to dimensionality reduction methods.

Use $L_1$ regularization to encourage sparsity such that only certain neurons fire.

$$J_{aug}(h) = J_{train} + \frac{\lambda}{m}\Omega(h)$$

$$J_{aug}(h) = \underbrace{||\Theta x - y||_2^2}_{\text{model fit}} + \underbrace{\lambda||\Theta||_1}_{\text{sparsity}} + \gamma||\Theta||_2^2$$

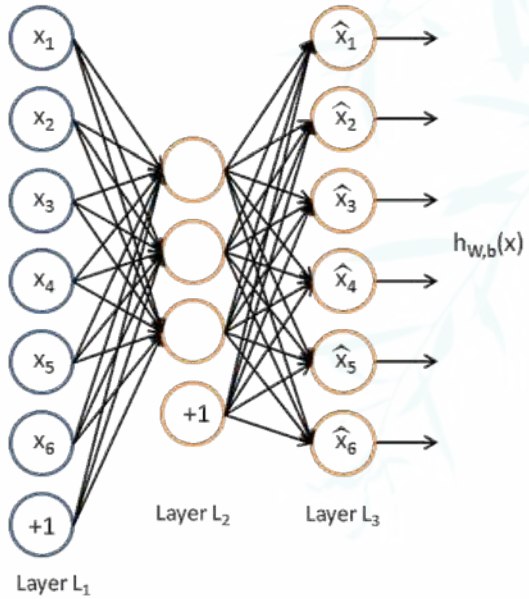
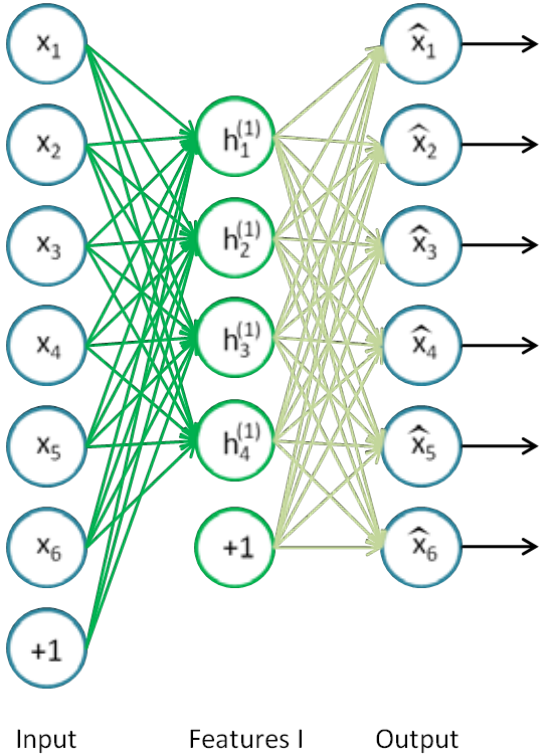


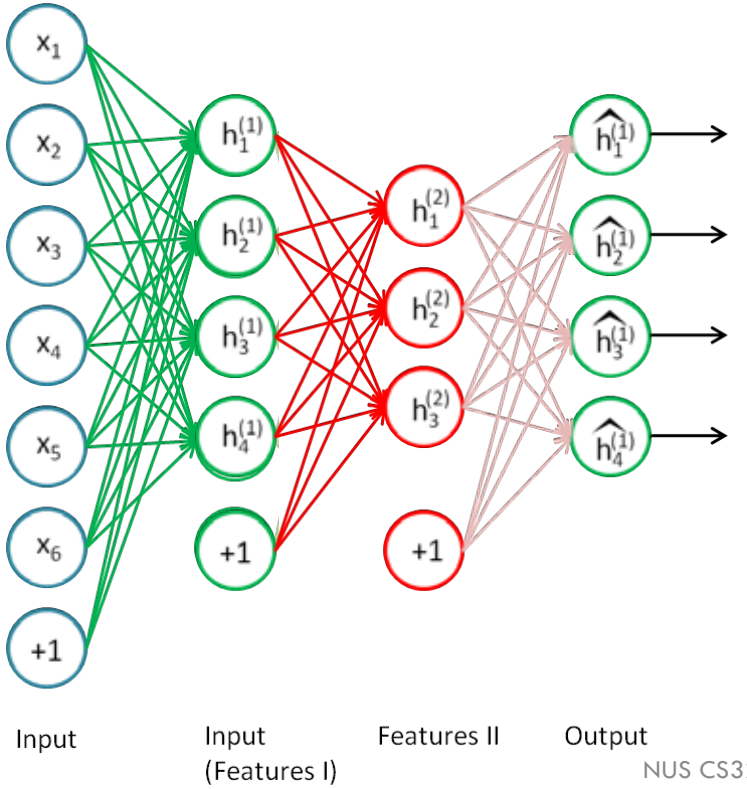



Image Credits: Stanford Deep Learning Website

# Train Hidden Layer 1



# Train Hidden Layers 2 thru $L$

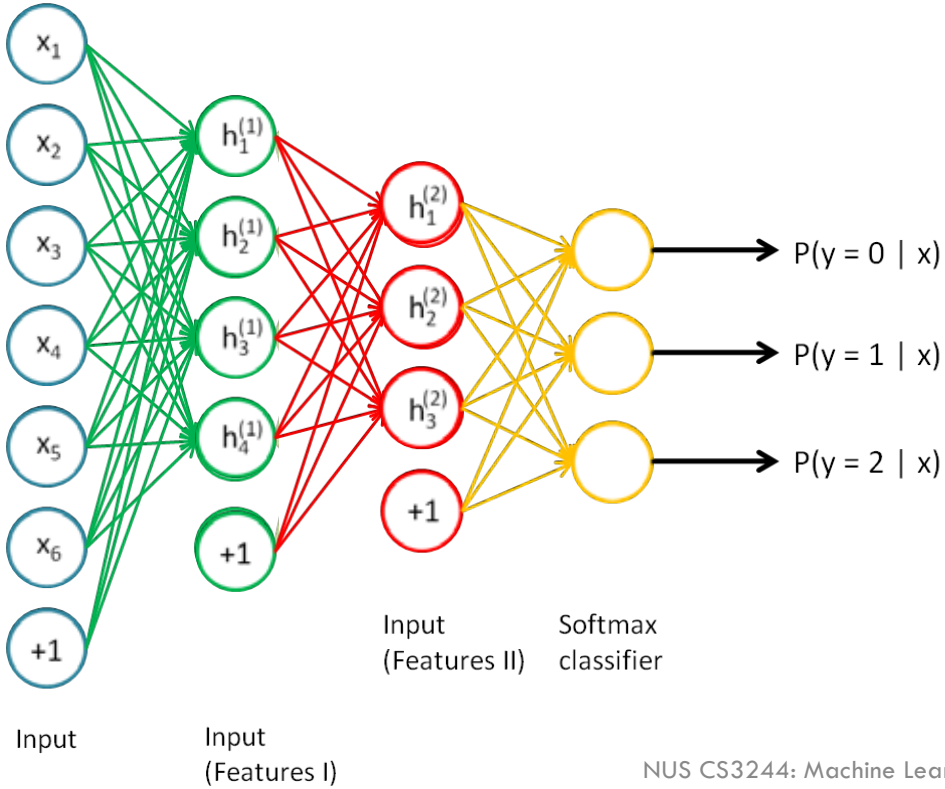



By discarding the output layer, we retain the wanted hidden layer's feature extraction

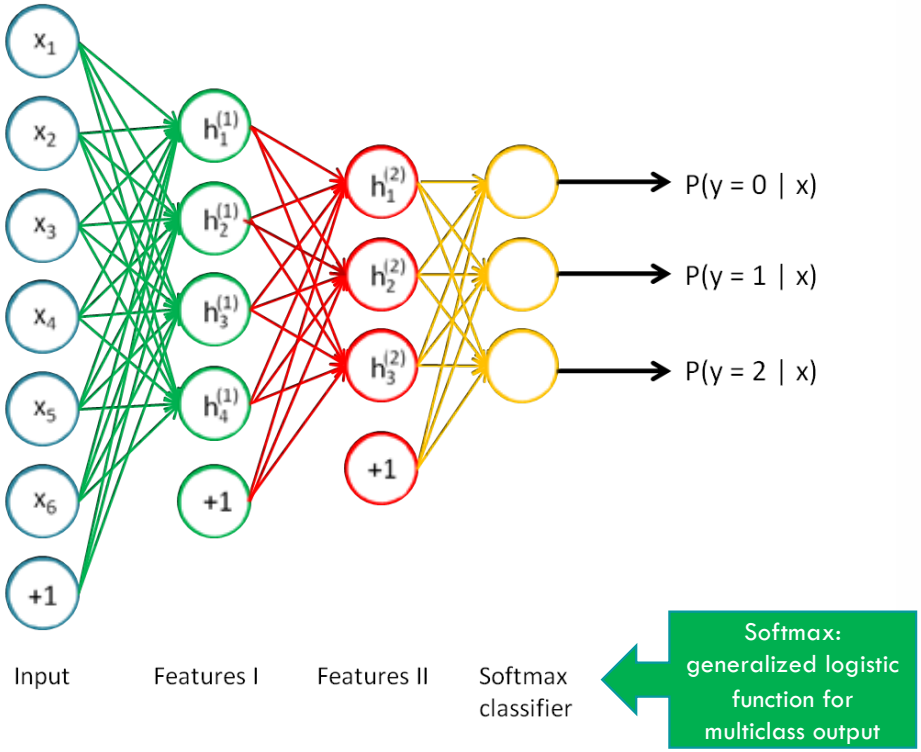
Then train the next hidden layer

Do this for all hidden layers

# Train Output Layer



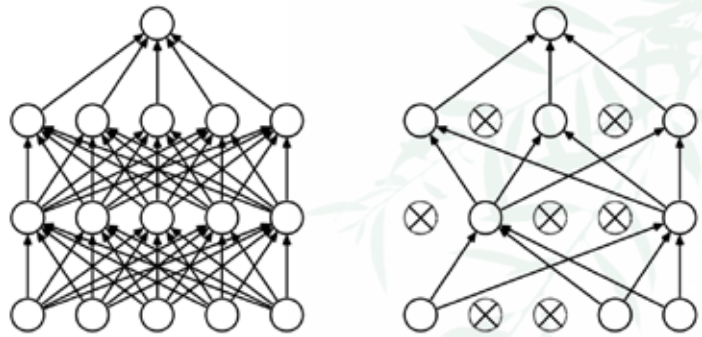
# Result: Pre-trained network




Then proceed using normal supervised NN training: feedforward, backprop to fine-tune results.

# Dropout

Still, DNNs can overfit quite badly.
Need more regularization.



(a) Standard Neural Net (b) After applying dropout.

Drop out $p = 0.5$ of the neurons when training for each example (minibatch).

- Training: neurons present with $P(p)$
- Testing: always use all units, weights $\leftarrow p\Theta$

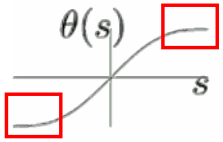
Point forwards to the Random Forest's feature projection idea

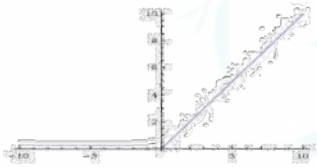
# Summary

Deep Learning ≡ Stacked units

3 important innovations:

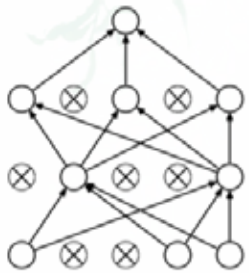
1. ReLU – Vanishing Gradient



2. Initialization – Pre-training via autoencoding

3. Dropout – Regularization



(b) After applying dropout.